

A Technical Introduction to Reinforcement learning

Section 1

Value function approaches attempt to find a policy that maximizes the discounted return by maintaining a set of estimates of expected discounted returns $E[\mathbb{E}[G] | s]$ for some policy (usually either the "current" [on-policy] or the optimal [off-policy] one). These methods rely on the theory of Markov decision processes, where optimality is defined in a sense stronger than the one above: A policy is optimal if it achieves the best-expected discounted return from any initial state (i.e., initial distributions play no role in this definition). Again, an optimal policy can always be found among stationary policies. To define optimality in a formal manner, define the state-value of a policy π by

Section 2

Partially supervised RL (PSRL) The costly exploration needed to learn an optimal policy can be reduced if some supervised data is available. This can be achieved for instance by learning a crude control policy and using this to initialize the Q table intelligently instead of with zeros.

Section 3

where G stands for the discounted return associated with following π from the initial state s . Defining $V^*(s)$ as the maximum possible state-value of $V^\pi(s)$, where π is allowed to change,

Section 4

A policy that achieves these optimal state-values in each state is called optimal. Clearly, a policy that is optimal in this sense is also optimal in the sense that it maximizes the expected discounted return, since $V^*(s) = \max_{\pi} E[G | s, \pi]$, where s is a state randomly sampled from the distribution μ of initial states (so $\mu(s) = \Pr(S_0 = s)$). Although state-values suffice to define optimality, it is useful to define action-values. Given a state s , an action a and a policy π , the action-value of the pair (s, a) under π is defined by

Section 5

The first problem is corrected by allowing the procedure to change the policy (at some or all states) before the values settle. This too may be problematic as it might prevent convergence. Most current algorithms do this, giving rise to

the class of generalized policy iteration algorithms. Many actor-critic methods belong to this category. The second issue can be corrected by allowing trajectories to contribute to any state-action pair in them. This may also help to some extent with the third problem, although a better solution when returns have high variance is Sutton's temporal difference (TD) methods that are based on the recursive Bellman equation. The computation in TD methods can be incremental (when after each transition the memory is changed and the transition is thrown away), or batch (when the transitions are batched and the estimates are computed once based on the batch). Batch methods, such as the least-squares temporal difference method, may use the information in the samples better, while incremental methods are the only choice when batch methods are infeasible due to their high computational or memory complexity. Some methods try to combine the two approaches. Methods based on temporal differences also overcome the fourth issue. Another problem specific to TD comes from their reliance on the recursive Bellman equation. Most TD methods have a so-called λ parameter ($0 \leq \lambda \leq 1$) that can continuously interpolate between Monte Carlo methods that do not rely on the Bellman equations and the basic TD methods that rely entirely on the Bellman equations. This can be effective in palliating this issue.

Section 6

where G now stands for the random discounted return associated with first taking action a in state s and following π , thereafter. The theory of Markov decision processes states that if π^* is an optimal policy, we act optimally (take the optimal action) by choosing the action from $Q^{\pi^*}(s, \cdot)$ with the highest action-value at each state, s . The action-value function of such an optimal policy (Q^{π^*}) is called the optimal action-value function and is commonly denoted by Q^* . In summary, the knowledge of the optimal action-value function alone suffices to know how to act optimally. Assuming full knowledge of the Markov decision process, the two basic approaches to compute the optimal action-value function are value iteration and policy iteration. Both algorithms compute a sequence of functions Q_k ($k = 0, 1, 2, \dots$) that converge to Q^* . Computing these functions involves computing expectations over the whole state-space, which is impractical for all but the smallest (finite) Markov decision processes. In reinforcement learning methods, expectations are approximated by averaging over samples and using function approximation techniques to cope with the need to represent value functions over large state-action spaces.

Section 7

A set of environment and agent states (the state space), S ; A set of actions (the action space), A , of the agent;

Section 8

that maximizes the expected cumulative reward. Formulating the problem as a Markov decision process assumes the agent directly observes the current environmental state; in this case, the problem is said to have full observability. If the agent only has access to a subset of states, or if the observed states are corrupted by noise, the agent is said to have partial observability, and formally the problem must be formulated as a partially observable Markov decision process. In both cases, the set of actions available to the agent can be restricted. For example, the state of an account balance could be restricted to be positive; if the current value of the state is 3 and the state transition attempts to reduce the value by 4, the transition will not be allowed. When the agent's performance is compared to that of an agent that acts optimally, the difference in performance yields the notion of regret. In order to act near optimally, the agent must reason about long-term consequences of its actions (i.e., maximize future rewards), although the immediate reward associated with this might be negative. Thus, reinforcement learning is particularly well-suited to problems that include a long-term versus short-term reward trade-off. It has been applied successfully to various problems, including energy storage, robot control, photovoltaic generators, backgammon, checkers, Go (AlphaGo), and autonomous driving systems. Two elements make reinforcement learning powerful: the use of samples to optimize performance, and the use of function approximation to deal with large environments. Thanks to these two key components, RL can be used in large environments in the following situations: